LLM Assignment 2 - Task 2

Linear Regression Using a Multi-Layer Neural Network From Scratch

1. Objective

The aim of this task is to implement a small multi-layer neural network from scratch and use it for predicting the median house value from the California Housing dataset. The main purpose was to understand how forward propagation, backpropagation and different optimizers work without using a pre-built neural network model.

The required network uses two input features, two hidden layers and one output neuron. Gradient Descent, Momentum and Adam were tested with learning rates 0.01 and 0.001. The assignment also contains two bonus parts: adding another hidden layer and applying L2 regularization.

2. Dataset and Preprocessing

The notebook uses sklearn.datasets.fetch_california_housing only to load the specified California Housing dataset. The two input features are MedInc (Median Income) and HouseAge (House Age), and the target is MedHouseVal (Median House Value). The loaded dataset contains 20,640 samples with two input features and one target.

A manual 80/20 split is performed using a reproducible seed of 42, producing 16,512 training samples and 4,128 testing samples. Feature normalization uses only the training-set mean and standard deviation, and the same training statistics are applied to the test set. The recorded training mean is [3.8716266, 28.70724322] and the training standard deviation is [1.90161359, 12.60605401].

$$X_{norm} = (X - mean) / standard\ deviation$$

3. Network Architecture

The required network is:

$$2 \rightarrow 5 \rightarrow 3 \rightarrow 1$$

The first hidden layer contains 5 neurons and the second hidden layer contains 3 neurons. Both hidden layers use ReLU activation. The output layer is linear because the task is regression.

Weights use He initialization, which is appropriate for ReLU hidden layers. Biases are initialized to zero.

4. Forward Propagation and Loss

Forward propagation is implemented as:

$$Z1 = XW1 + b1$$
$$A1 = ReLU(Z1)$$
$$Z2 = A1W2 + b2$$
$$A2 = ReLU(Z2)$$
$$y\_hat = A2W3 + b3$$

The mean squared error is:

$$MSE = mean((y - y\_hat)^2)$$

5. Backpropagation

The notebook computes the MSE derivative as 2(y_hat - y)/m, where m is the number of training samples. The resulting output gradient is propagated through the linear output layer and the two ReLU hidden layers. The weight gradients are computed with matrix multiplications and the bias gradients are obtained by summing over the training examples. The implementation therefore performs the complete backward pass manually.

6. Optimizers

Three required optimizers are implemented:

• Gradient Descent: theta_(t+1) = theta_t - alpha*g.

• Momentum: v = beta*v_(t-1) + g, followed by theta_(t+1) = theta_t - alpha*v.

• Adam: maintains first and second moment estimates, applies bias correction, and uses adaptive parameter updates.

Each experiment is trained for 1,000 epochs.

7. Required Experiments and Results

Optimizer	Learning Rate	Test MSE
Gradient Descent	0.01	0.68558
Gradient Descent	0.001	1.90709
Momentum	0.01	0.65785
Momentum	0.001	0.83841
Adam	0.01	0.65285
Adam	0.001	0.69531

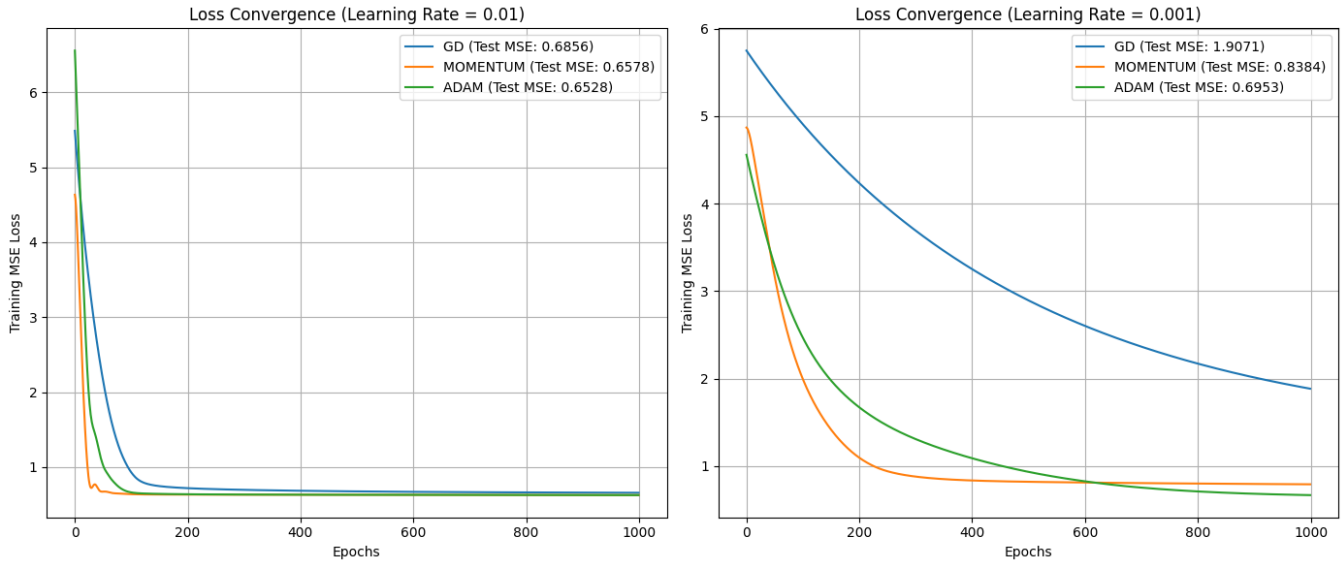
7.1. Required Results — Discussion

Among the required experiments, Adam with learning rate 0.01 gives the lowest test MSE, 0.65285. Momentum with learning rate 0.01 is the second-best required configuration with test MSE 0.65785. Gradient Descent at learning rate 0.001 gives the highest test MSE, 1.90709.

For all three optimizers in this experiment, learning rate 0.01 produces a lower test MSE than 0.001. The results therefore show that the larger tested learning rate is more effective for this particular network, dataset split and 1,000-epoch training setup.

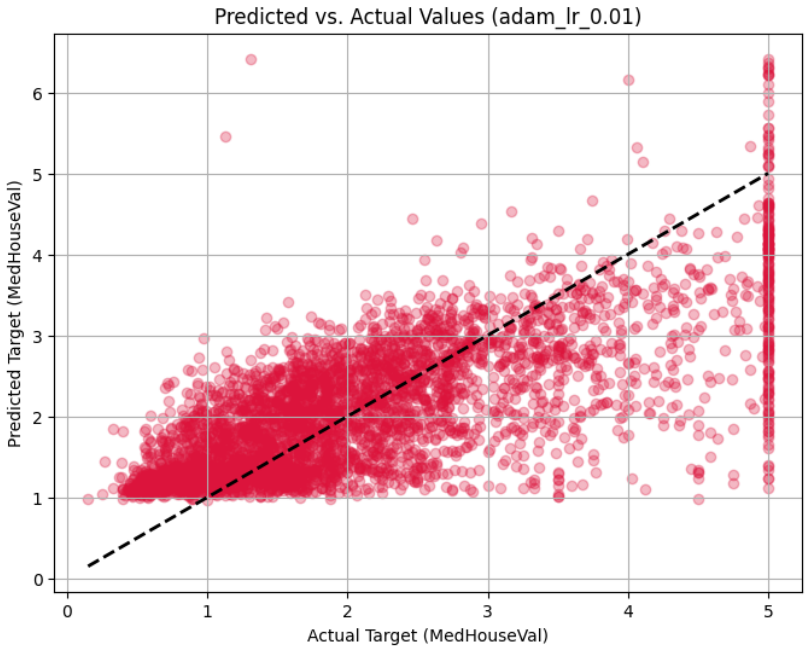
8. Training-Loss Curves and Prediction Analysis

The notebook records the training MSE at every epoch and produces training-loss curves for the required experiments. It also evaluates the trained models on the held-out test set and generates a predicted-versus-actual plot for the best required configuration, Adam with learning rate 0.01.



Training-loss convergence for the required optimizer and learning-rate experiments.

8. Training-Loss Curves and Prediction Analysis



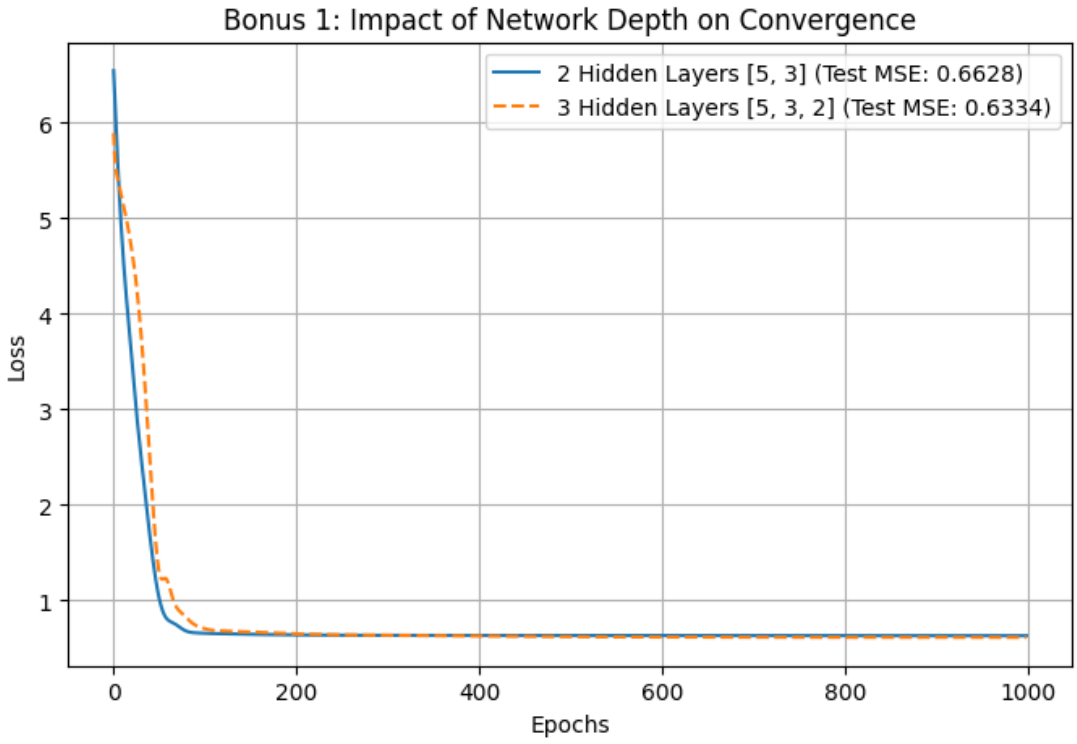
Predicted versus actual values for Adam with learning rate 0.01.

9. Bonus 1 — Additional Hidden Layer

Bonus 1 adds a third hidden layer containing 2 neurons, changing the architecture from 2 -> 5 -> 3 -> 1 to 2 -> 5 -> 3 -> 2 -> 1. The additional hidden layer also uses ReLU. The notebook compares this deeper architecture using Adam at learning rate 0.01.

Architecture	LR	Test MSE
Original 2->5->3->1	0.01	0.66279
Bonus 2->5->3->2->1	0.01	0.63341

At learning rate 0.01, the additional hidden layer improves test MSE from 0.66279 to 0.63341. Thus, the additional layer improves the result in this experiment.



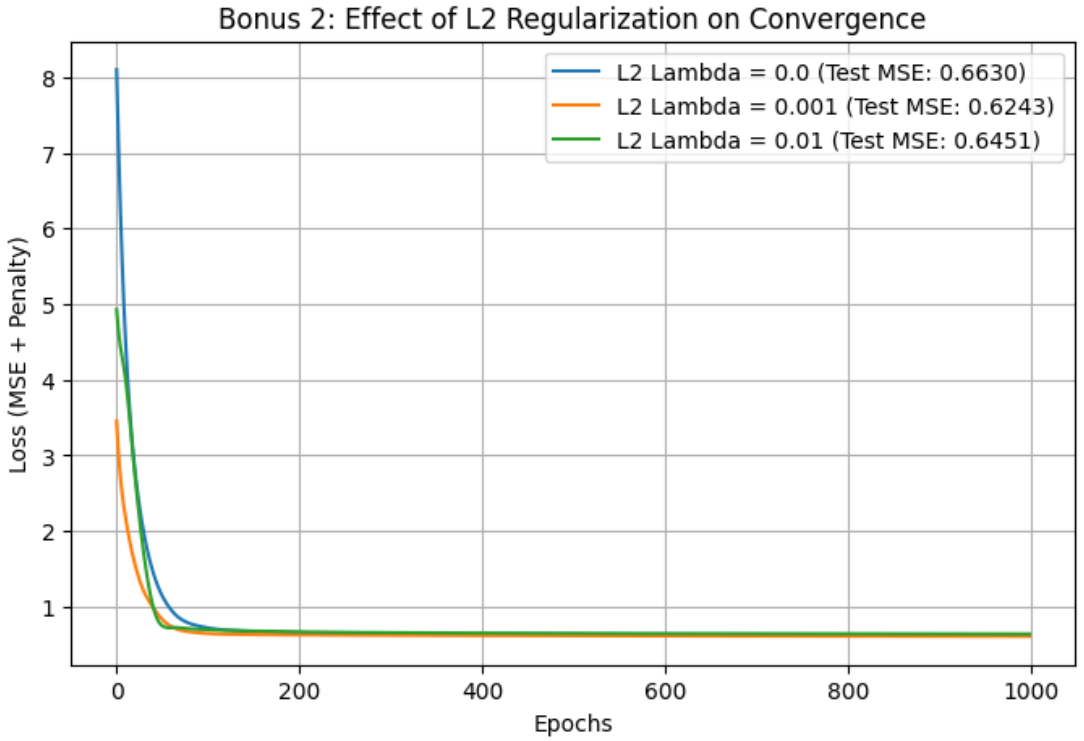
Bonus 1 training-loss comparison between the original and deeper architectures.

10. Bonus 2 — L2 Regularization

Bonus 2 adds L2 regularization to the original network. The notebook compares lambda = 0, lambda = 0.001 and lambda = 0.01. The regularization penalty is applied to the network weights; biases are not regularized.

Lambda	Test MSE
0.000	0.66296
0.001	0.62430
0.010	0.64513

With lambda = 0.001, the test MSE decreases from 0.66296 to 0.62430. In this experiment, lambda = 0.001 gives the best result among the tested regularization values.



Bonus 2 training-loss comparison for the tested L2 regularization values.

11. Overall Comparison

Experiment	Best Test MSE	Configuration
Required experiments	0.65285	Adam, LR = 0.01
Bonus 1	0.63341	2->5->3->2->1, Adam, LR = 0.01
Bonus 2	0.62430	L2 lambda = 0.001, Adam, LR = 0.01

The lowest test MSE recorded in the notebook is 0.62430 from Bonus 2 with L2 lambda = 0.001. The deeper Bonus 1 architecture achieves 0.63341 at learning rate 0.01, improving on its original two-hidden-layer comparison.

12. Conclusion

The assignment demonstrates that a neural network for regression can be constructed entirely from basic numerical operations. The required 2 -> 5 -> 3 -> 1 network performs forward propagation, backpropagation, and optimization without using a pre-built neural-network or optimizer library. Among the required configurations, Adam with learning rate 0.01 achieves the best test MSE of 0.65285. The bonus experiments show that the additional hidden layer improves the tested Adam result and L2 regularization with lambda = 0.001 gives the lowest test MSE recorded in the notebook.

13. Implementation Verification

The notebook records the dataset loading, manual split, training-only normalization, required network architecture, manual backpropagation, three required optimizers, both learning rates, 1,000 epochs, training loss tracking, test MSE, training-loss plots, predicted-versus-actual plots, and both requested bonus experiments.

The executed notebook also reports the corresponding numerical results used in this report.